



RemoteOps Implementation Report

This report explains the implemented C Agent and Controller, the design decisions and the recorded tests. All screenshots belong to this project. Mandatory features passed the documented tests; no optional extension is claimed.

Student	Heishan
Registration No.	IT24100639
Environment	Windows and CentOS 10 VirtualBox
Repository	https://github.com/IT24100639/IE3090-RemoteOps
Submission Archive	IE3090_IT24100639.zip

Table of Contents

1. Introduction and Personalisation	4
1.1 Personalised Settings	4
1.2 Configuration and Listening Port Evidence	5
2. Architecture and Concurrent Connections.....	6
2.1 Agent and Controller Responsibilities	6
2.2 Socket Setup and Thread Creation	7
2.3 Five Simultaneous Controllers.....	8
3. TCP Protocol, Authentication and System Commands	11
3.1 Command Framing and Response Formats	11
3.2 Partial Sends and Exact-Byte Reception.....	12
3.3 Authentication and Access Control	14
3.4 System Information	15
3.5 Process Snapshot.....	16
3.6 Restricted Command Execution	18
4. File Transfers and Integrity	20
4.1 Binary Protocol and Safe Storage.....	20
4.2 Upload Implementation	20
4.3 Download Implementation	22
4.4 Transfer Execution and Content Integrity	24
4.5 Rejected and Interrupted Transfers	25
5. UDP Monitoring and Session Cleanup	27
5.1 Per-Session UDP Lifecycle	27
5.2 Monitoring Worker and Stop Coordination	27
5.3 Monitoring Execution.....	29
5.4 Independent Monitoring Sessions	30
6. Logging and Error Handling	31
6.1 Timestamped Event Records.....	31
6.2 Error Behaviour	32
7. Build and Testing Summary	33
7.1 Final Clean Build	34
7.2 TCP Stream Robustness.....	34
8. Incremental Development and AI Assistance	35
8.1 Meaningful Development Milestones.....	35
8.2 Design Diary and Reflection	35
8.3 AI Assistance and Practical Verification	36
9. Assumptions, Limitations and Future Improvements	37
9.1 Operating Assumptions.....	37
9.2 Implementation Limitations.....	37
9.3 Optional Extension Status and Future Work	37
10. Conclusion	38

List of Figures

Figure 1. Personalised Configuration	5
Figure 2. Agent TCP Listener	5
Figure 3. Agent Startup	5
Figure 4. TCP Socket Setup.....	7
Figure 5. Accept and Client Thread Creation.....	8
Figure 6. Five TCP Connections	9
Figure 7. Five C Controller Results	10
Figure 8. Send All Bytes.....	12
Figure 9. Receive Exact Bytes	13
Figure 10. Read a Command Line	13
Figure 11. SID Response Helper.....	14
Figure 12. Authentication Handler	14
Figure 13. Authentication and Session Close	15
Figure 14. SYSINFO Handler	15
Figure 15. SYSINFO Execution	16
Figure 16. LISTPROC Traversal.....	16
Figure 17. LISTPROC Output Construction	17
Figure 18. LISTPROC Execution.....	17
Figure 19. EXEC: Time, Uptime and Disk	18
Figure 20. EXEC: Hostname, User and Rejection	18
Figure 21. Allowed EXEC Commands	19
Figure 22. Rejected EXEC Requests	19
Figure 23. PUT Request Validation	20
Figure 24. PUT Payload Reception.....	21
Figure 25. PUT Completion	22
Figure 26. GET File Validation.....	22
Figure 27. GET Header and Payload.....	23
Figure 28. GET Completion.....	23
Figure 29. Text Upload and Download.....	24
Figure 30. Text Transfer Integrity	24
Figure 31. Binary Transfer Integrity	24
Figure 32. Empty File Transfer.....	25
Figure 33. Personalised Storage and Log	25
Figure 34. Invalid GET and Missing File.....	26
Figure 35. Upload Error Tests.....	26
Figure 36. UDP Monitoring Worker	27
Figure 37. UDP Interval and Stop Coordination	28
Figure 38. MONITOR Commands and Disconnect	28
Figure 39. UDP Monitoring Stream.....	29
Figure 40. Further Monitoring Updates.....	29
Figure 41. Monitoring Stop and TCP Continuity	30
Figure 42. Monitoring Isolation Tests	30
Figure 43. Logging Implementation	31
Figure 44. Timestamped Log Evidence	31
Figure 45. Final Clean Build	34
Figure 46. TCP Framing Tests	34

1. Introduction and Personalisation

RemoteOps consists of an Agent and Controller written in C using BSD sockets. TCP carries authenticated commands and binary file transfers. UDP carries periodic system statistics. The recorded tests were performed inside my CentOS 10 VM through Windows Terminal SSH access.

1.1 Personalised Settings

Item	Derived value
Registration and numeric part	IT24100639
TCP listening port	First four digits 2410; $7000 + 2410 = 9410$
SID tag	Last four digits 0639 reversed to 9360; SID:9360
Authentication token	OPS- followed by 0639 gives OPS-0639
Source and build names	Last three digits 639: agent_639.c, controller_639.c, Makefile_639
Log filename	remoteops_IT24100639.log
Agent storage	./agentfiles/IT24100639/<filename>
Submission archive	IE3090_IT24100639.zip
Chosen operating values	UDP interval 2 seconds; transfer chunk 4096 bytes; maximum file 104857600 bytes (100 MiB)

Repository: <https://github.com/IT24100639/IE3090-RemoteOps>

Both programs run inside the same CentOS VM for the recorded tests. The Controller connects to 127.0.0.1:9410. Windows host port 2222 forwards to guest SSH port 22; it is separate from the Agent listening port.

1.2 Configuration and Listening Port Evidence

```
heishan@localhost:~/IE3090_R x heishan@localhost:~/IE3090_ x
#define REMOTEOPS_CONFIG_H
#define REMOTEOPS_CONFIG_H

#define REG_NUMBER "IT24100639"
#define DEFAULT_PORT 9410
#define SID_TAG "SID:9360"
#define AUTH_TOKEN "OPS-0639"

#define LOG_FILE "remoteops_IT24100639.log"
#define STORAGE_DIR "./agentfiles/IT24100639"

#define MAX_LINE 4096
#define FILE_CHUNK_SIZE 4096
#define MAX_FILE_SIZE (100ULL * 1024ULL * 1024ULL)
#define MONITOR_INTERVAL_SEC 2

#endif
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 1. Personalised Configuration

remoteops_config.h defines the registration, calculated port, SID, token, log/storage names and operating limits.

```
heishan@localhost:~/IE3090_R x heishan@localhost:~/IE3090_ x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\heish> ssh -p 2222 heishan@127.0.0.1
Web console: https://localhost:9090/ or https://10.0.2.15:9090/

Last login: Fri Oct 2 20:54:51 2026 from 10.0.2.2
heishan@localhost:~$ cd ~/IE3090_RemoteOps
heishan@localhost:~/IE3090_RemoteOps$ ss -ltnp 'sport = :9410'
State Recv-Q Send-Q Local Address:Port Peer Address:Port Process
LISTEN 0 16 0.0.0.0:9410 0.0.0.0:* users:(("agent_639",pid=4921,fd=3))
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 2. Agent TCP Listener

The ss output shows the Agent listening on the personalised TCP port 9410.

```
heishan@localhost:~/IE3090_ x + v
RemoteOps Agent IT24100639 listening on TCP port 9410
Log file: remoteops_IT24100639.log
|
```

Figure 3. Agent Startup

The running Agent announces IT24100639, TCP port 9410 and remoteops_IT24100639.log.

2. Architecture and Concurrent Connections

2.1 Agent and Controller Responsibilities

The Agent is one C process with a listening TCP socket. Its main loop accepts connections and creates one detached pthread per client. Each client thread owns a connected socket, authentication state and monitoring state. This allows one client to wait for input without blocking other sessions. The recorded five-Controller test demonstrated simultaneous authenticated connections and successful QUIT responses.

An authenticated MONITOR START request creates a separate UDP worker for that session. The worker obtains system statistics and sends a datagram every two seconds to the TCP peer address and requested UDP port. A mutex and condition variable coordinate stopping; the session joins the monitoring worker before closing its resources. Detached TCP client threads release their thread resources when they exit.

The Controller creates a TCP connection using `socket()`, `inet_pton()` and `connect()`. It accepts commands from standard input and uses `poll()` to watch user input and the monitoring UDP socket. It calculates upload lengths and saves downloaded files under `./downloads`. UDP output can appear while it accepts commands; it checks the Agent IP and expected message SID.

Shared resources are the registration-specific storage directory and append-only log. A log mutex protects complete entries from interleaving. Uploads use separate temporary files and a rename after completion. If two sessions finish uploads to the same filename, the last successful rename determines the final contents; this implementation does not provide per-file locking.

Component	Responsibility
<code>agent_639.c</code>	TCP listener, client threads, authentication and command dispatch; SYSINFO, LISTPROC and EXEC
<code>controller_639.c</code>	TCP connection, interactive commands, upload/download handling and UDP reception
<code>remoteops_config.h</code>	Personalisation, limits and interval
<code>remoteops_io.h</code>	Partial send loops, exact-byte reception and newline command reception
<code>remoteops_files.h</code>	Filename/size validation and Agent transfer handlers
<code>remoteops_monitor.h</code>	Per-session UDP worker and lifecycle
<code>remoteops_log.h</code>	Timestamped, mutex-protected event records
<code>Makefile_639</code>	Compile both programs directly with GCC and pthread support

This model is straightforward for a small lab workload and uses the BSD socket layer directly. It has no maximum client-thread count or application idle timeout, so the concurrency test establishes five working clients rather than unlimited capacity.

2.2 Socket Setup and Thread Creation

```
heishan@localhost:~/IE3090_R × heishan@localhost:~/IE3090_ ×  
  
int server_fd = socket(AF_INET, SOCK_STREAM, 0);  
if (server_fd == -1) {  
    perror("socket");  
    return EXIT_FAILURE;  
}  
  
int reuse = 1;  
if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR,  
               &reuse, sizeof(reuse)) == -1) {  
    perror("setsockopt");  
    close(server_fd);  
    return EXIT_FAILURE;  
}  
  
struct sockaddr_in address = {0};  
address.sin_family = AF_INET;  
address.sin_port = htons(DEFAULT_PORT);  
address.sin_addr.s_addr = htonl(INADDR_ANY);  
  
if (bind(server_fd, (struct sockaddr *)&address,  
         sizeof(address)) == -1) {  
    perror("bind");  
    close(server_fd);  
    return EXIT_FAILURE;  
}  
  
if (listen(server_fd, 16) == -1) {  
    perror("listen");  
    close(server_fd);  
    return EXIT_FAILURE;  
}  
  
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 4. TCP Socket Setup

socket(), SO_REUSEADDR, bind() and listen() configure an IPv4 listener on all VM interfaces at DEFAULT_PORT.

```
heishan@localhost:~/IE3090_R  X  heishan@localhost:~/IE3090_  X  Windows PowerShell

for (;;) {
    struct sockaddr_in peer = {0};
    socklen_t peer_length = sizeof(peer);

    int client_fd = accept(server_fd,
                          (struct sockaddr *)&peer,
                          &peer_length);

    if (client_fd == -1) {
        if (errno == EINTR) {
            continue;
        }
        perror("accept");
        log_event(-1, "ERROR", "accept failed: %s",
                  strerror(errno));
        break;
    }

    char peer_ip[INET_ADDRSTRLEN] = "unknown";
    inet_ntop(AF_INET, &peer.sin_addr,
              peer_ip, sizeof(peer_ip));

    printf("Connection from %s:%u\n",
           peer_ip, (unsigned)ntohs(peer.sin_port));
    fflush(stdout);

    log_event(client_fd, "CONNECT", "TCP connection accepted");

    int *argument = malloc(sizeof(*argument));
    if (argument == NULL) {
        perror("malloc");
        log_event(client_fd, "ERROR", "client allocation failed");
        close(client_fd);
        continue;
    }
    *argument = client_fd;

    pthread_t thread;
    error = pthread_create(&thread, &attributes,
                          handle_client, argument);

    if (error != 0) {
        fprintf(stderr, "pthread_create: %s\n", strerror(error));
        log_event(client_fd, "ERROR",
                  "client thread creation failed: %s",
                  strerror(error));
        free(argument);
        close(client_fd);
    }
}
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 5. Accept and Client Thread Creation

accept() creates a connected socket; a separately allocated argument is handed to pthread_create(). Failure paths free the argument and close the socket.

2.3 Five Simultaneous Controllers

The connection listing and saved Controller replies provide complementary evidence: five sockets were established together, and each C Controller authenticated and later received its BYE response.

```

heishan@localhost:~$ cd ~/IE3090_RemoteOps
heishan@localhost:~/IE3090_RemoteOps$ test_dir=$(mktemp -d /tmp
/remoteops-five-XXXXXX)
for i in 1 2 3 4 5
do
    { printf 'AUTH OPS-0639\n'; sleep 180; printf 'QUIT\n'; } |
        ./controller_639 > "$test_dir/controller_${i}.txt" 2>&1 &
done
[1] 5393
[2] 5395
[3] 5398
[4] 5403
[5] 5405
heishan@localhost:~/IE3090_RemoteOps$ ss -tnp state established
'sport = :9410'
Recv-Q    Send-Q      Local Address:Port      Peer Address:Port
Process
0          0          127.0.0.1:9410          127.0.0.1:35826
users:(("agent_639",pid=5283,fd=6))
0          0          127.0.0.1:9410          127.0.0.1:35804
users:(("agent_639",pid=5283,fd=4))
0          0          127.0.0.1:9410          127.0.0.1:35830
users:(("agent_639",pid=5283,fd=7))
0          0          127.0.0.1:9410          127.0.0.1:35846
users:(("agent_639",pid=5283,fd=8))
0          0          127.0.0.1:9410          127.0.0.1:35820
users:(("agent_639",pid=5283,fd=5))
heishan@localhost:~/IE3090_RemoteOps$

```

Figure 6. Five TCP Connections

The connection listing shows five established sessions on Agent port 9410.

```
heishan@localhost:~ x heishan@localhost:~ x + v - □ ×
0 0 127.0.0.1:9410 127.0.0.1:35820
users:(("agent_639",pid=5283,fd=5))
heishan@localhost:~/IE3090_RemoteOps$ cat "$test_dir"/controller_*.txt
Connected to Agent at 127.0.0.1:9410
Enter a command, then press Enter.
remoteops> OK AUTHENTICATED SID:9360
remoteops> OK BYE SID:9360
Connected to Agent at 127.0.0.1:9410
Enter a command, then press Enter.
remoteops> OK AUTHENTICATED SID:9360
remoteops> OK BYE SID:9360
Connected to Agent at 127.0.0.1:9410
Enter a command, then press Enter.
remoteops> OK AUTHENTICATED SID:9360
remoteops> OK BYE SID:9360
Connected to Agent at 127.0.0.1:9410
Enter a command, then press Enter.
remoteops> OK AUTHENTICATED SID:9360
remoteops> OK BYE SID:9360
Connected to Agent at 127.0.0.1:9410
Enter a command, then press Enter.
remoteops> OK AUTHENTICATED SID:9360
remoteops> OK BYE SID:9360
[1] Done { printf 'AUTH OPS-0639\n'; sleep
180; printf 'QUIT\n'; } | ./controller_639 > "$test_dir/controller_$i.txt" 2>&1
[2] Done { printf 'AUTH OPS-0639\n'; sleep
180; printf 'QUIT\n'; } | ./controller_639 > "$test_dir/controller_$i.txt" 2>&1
[3] Done { printf 'AUTH OPS-0639\n'; sleep
180; printf 'QUIT\n'; } | ./controller_639 > "$test_dir/controller_$i.txt" 2>&1
[4]- Done { printf 'AUTH OPS-0639\n'; sleep
180; printf 'QUIT\n'; } | ./controller_639 > "$test_dir/controller_$i.txt" 2>&1
[5]+ Done { printf 'AUTH OPS-0639\n'; sleep
180; printf 'QUIT\n'; } | ./controller_639 > "$test_dir/controller_$i.txt" 2>&1
```

Figure 7. Five C Controller Results

The saved C Controller outputs show authentication and BYE responses for the five concurrent sessions.

3. TCP Protocol, Authentication and System Commands

3.1 Command Framing and Response Formats

TCP is a byte stream. A command ends in a newline; `recv_line()` reads only through that newline. File transfers use a text header followed by an exact-length binary payload. `send_all()` and `recv_exact()` loop over partial socket operations and retry `EINTR`. They do not assume one send or `recv` is one message. Normal responses and file headers append `SID:9360` and a newline; raw file bytes are not modified to append a `SID`.

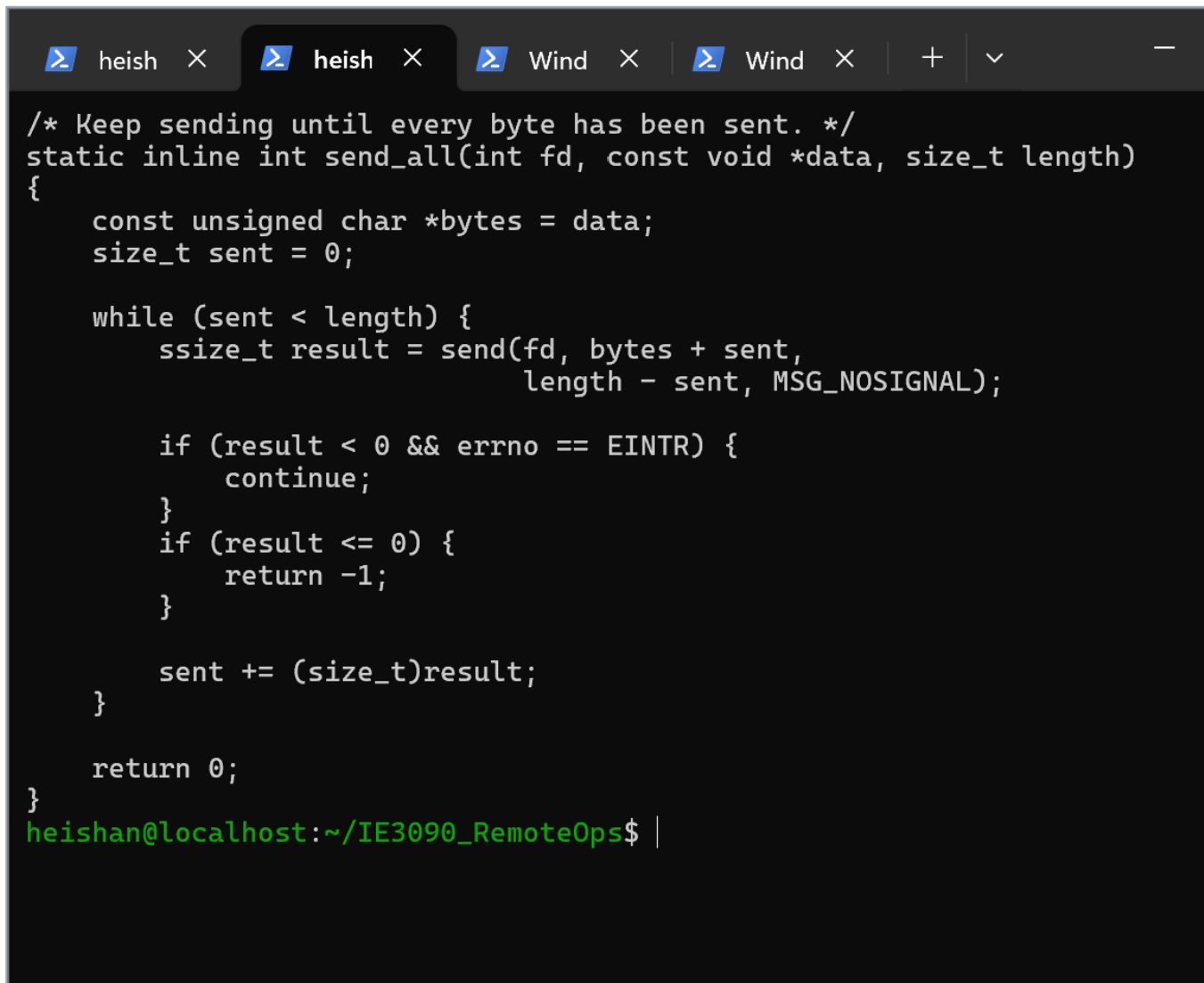
Request	Behaviour and response form
AUTH OPS-0639	OK AUTHENTICATED <code>SID:9360</code> ; wrong token returns <code>ERR 001 AUTH_FAILED</code> and clears authentication
SYSINFO	OK SYSINFO <code><load> <memory> <uptime></code> <code>SID:9360</code>
LISTPROC	OK PROCS <code><name/PID entries></code> <code>SID:9360</code> ; entries are comma-separated
EXEC DATE / UPTIME / DISKFREE / HOSTNAME / WHOAMI	OK EXEC_RESULT <code><result></code> <code>SID:9360</code> ; only exact allowed names are accepted
PUT <filename> <size> then payload	Receive exactly <code><size></code> bytes; OK FILE_RECEIVED <code><filename></code> <code>SID:9360</code> after complete storage
GET <filename>	OK FILE_SEND <code><filename> <size></code> <code>SID:9360</code> then exactly <code><size></code> raw bytes
MONITOR START <udp_port>	OK MONITOR_STARTED <code>SID:9360</code> ; periodic UDP SYSINFO datagrams
MONITOR STOP	OK MONITOR_STOPPED <code>SID:9360</code> ; stop only this session
QUIT	OK BYE <code>SID:9360</code> ; stop monitoring and close this client

Commands other than `AUTH` require authentication. A failed reauthentication also stops monitoring. Ports must be decimal values from 1 to 65535. Unsupported commands, extra `EXEC` arguments and shell strings are rejected rather than executed.

`SYSINFO` uses Linux `sysinfo()`. Load is the one-minute load average, not a CPU percentage. Memory is (total RAM minus free RAM) multiplied by `mem_unit` and expressed in MiB, so caching is included. Uptime is seconds. `LISTPROC` reads `/proc/<pid>/comm`, returns up to 20 readable entries, skips processes that disappear, and replaces control/special characters to keep the response safe and bounded.

`EXEC` invokes C APIs rather than a shell: local time for `DATE`, `sysinfo` for `UPTIME`, `statvfs` on the project filesystem for `DISKFREE`, `uname` for `HOSTNAME` and `getpwuid_r` for the effective user in `WHOAMI`. `DISKFREE` is space available to the Agent user. No arbitrary command execution is supported.

3.2 Partial Sends and Exact-Byte Reception



```
/* Keep sending until every byte has been sent. */
static inline int send_all(int fd, const void *data, size_t length)
{
    const unsigned char *bytes = data;
    size_t sent = 0;

    while (sent < length) {
        ssize_t result = send(fd, bytes + sent,
                              length - sent, MSG_NOSIGNAL);

        if (result < 0 && errno == EINTR) {
            continue;
        }
        if (result <= 0) {
            return -1;
        }

        sent += (size_t)result;
    }

    return 0;
}
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 8. Send All Bytes

send_all() advances by the actual number sent, retries EINTR, and returns an error on failed sends. MSG_NOSIGNAL avoids SIGPIPE termination.

```
heish x heish x Wind x Wind x + v
/* Receive exactly length bytes.
   Return 0 on success or -1 on error/premature disconnect. */
static inline int recv_exact(int fd, void *data, size_t length)
{
    unsigned char *bytes = data;
    size_t received = 0;

    while (received < length) {
        ssize_t result = recv(fd, bytes + received,
                               length - received, 0);

        if (result < 0 && errno == EINTR) {
            continue;
        }
        if (result < 0) {
            return -1;
        }
        if (result == 0) {
            errno = ECONNRESET;
            return -1;
        }

        received += (size_t)result;
    }

    return 0;
}
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 9. Receive Exact Bytes

recv_exact() advances by the actual number received and fails on premature disconnect, preserving exact file boundaries.

```
heish x heish x Wind x Wind x + v -
/* Return 1 for a line, 0 for clean EOF, -1 for an error,
   or -2 if the line exceeds the buffer capacity.
   Reading one byte at a time leaves file payload bytes untouched. */
static inline int recv_line(int fd, char *line, size_t capacity)
{
    size_t used = 0;

    if (capacity == 0) {
        return -2;
    }

    for (;;) {
        char character;
        ssize_t result = recv(fd, &character, 1, 0);

        if (result < 0 && errno == EINTR) {
            continue;
        }
        if (result < 0) {
            return -1;
        }
        if (result == 0) {
            return used == 0 ? 0 : -1;
        }
        if (character == '\n') {
            line[used] = '\0';
            return 1;
        }
        if (used >= capacity - 1) {
            return -2;
        }

        line[used++] = character;
    }
}
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 10. Read a Command Line

recv_line() reads through newline without consuming following payload bytes and rejects an overlong line.

```
heishan@localhost:~/IE3090_RemoteOps$ |
```

```
/* Every TCP response ends with the personalised SID and newline. */
static int reply(int fd, const char *message)
{
    char response[MAX_LINE];
    int length = snprintf(response, sizeof(response),
                          "%s %s\n", message, SID_TAG);

    if (length < 0 || (size_t)length >= sizeof(response)) {
        log_event(fd, "ERROR", "response exceeded buffer capacity");
        return -1;
    }

    int result = send_all(fd, response, (size_t)length);
    log_event(fd, "RESPONSE", "send=%s message=%s",
              result == 0 ? "ok" : "failed", message);
    return result;
}
```

Figure 11. SID Response Helper

reply() appends SID_TAG and newline, checks buffer capacity, sends the full response and records its outcome.

3.3 Authentication and Access Control

```
heishan@localhost:~/IE3090_R x heishan@localhost:~/IE3090_ x Windows PowerShell
```

```
if (strncmp(line, "AUTH ", 5) == 0 ||
    strcmp(line, "AUTH") == 0) {
    log_event(fd, "COMMAND", "AUTH [redacted]");
} else {
    log_event(fd, "COMMAND", "%s", line);
}

const char *response;

if (strncmp(line, "AUTH ", 5) == 0) {
    if (strcmp(line + 5, AUTH_TOKEN) == 0) {
        authenticated = 1;
        response = "OK AUTHENTICATED";
        log_event(fd, "AUTH", "result=success");
    } else {
        authenticated = 0;
        monitor_stop(&monitor);
        response = "ERR 001 AUTH_FAILED";
        log_event(fd, "AUTH", "result=failed monitoring=stopped");
    }
} else if (!authenticated) {
    if (reply(fd, "ERR 003 AUTH_REQUIRED") == -1) {
        break;
    }

    /* A PUT payload cannot be treated as subsequent commands. */
    if (strncmp(line, "PUT ", 4) == 0) {
        log_event(fd, "PUT_REJECTED",
                  "authentication required; closing connection");
        break;
    }
    continue;
}
```

```
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 12. Authentication Handler

Each client has its own authentication state. Exact token matching enables commands; failure clears authentication and stops monitoring. Other commands are gated.

```

heishan@localhost:~/IE3090_RemoteOps$ gcc -std=c11 -Wall -Wextra -Wpedantic -O2 controller_639.c -o controller_639
heishan@localhost:~/IE3090_RemoteOps$ ./controller_639
Connected to Agent at 127.0.0.1:9410
Enter a command, then press Enter.
remoteops> SYSINFO
ERR 003 AUTH_REQUIRED SID:9360
remoteops> AUTH wrong-token
ERR 001 AUTH_FAILED SID:9360
remoteops> AUTH wrong-token
ERR 001 AUTH_FAILED SID:9360
remoteops> ERR 001 AUTH_FAILED SID:9360
ERR 003 AUTH_REQUIRED SID:9360
remoteops> AUTH OPS-0639
OK AUTHENTICATED SID:9360
remoteops> QUIT
OK BYE SID:9360
heishan@localhost:~/IE3090_RemoteOps$ |

```

Figure 13. Authentication and Session Close

The actual exchange includes rejected authentication, successful AUTH and QUIT replies with the SID. Accidental input remains visible in the original capture.

3.4 System Information

```

static int send_sysinfo(int fd)
{
    struct sysinfo information;

    if (sysinfo(&information) == -1) {
        return reply(fd, "ERR 007 SYSINFO_FAILED");
    }

    double load = (double)information.loads[0] / 65536.0;
    double memory =
        ((double)information.totalram - (double)information.freeram) *
        (double)information.mem_unit / (1024.0 * 1024.0);

    char message[256];
    snprintf(message, sizeof(message),
             "OK SYSINFO %.2f %.2f %ld",
             load, memory, information.uptime);

    return reply(fd, message);
}
heishan@localhost:~/IE3090_RemoteOps$ |

```

Figure 14. SYSINFO Handler

Linux sysinfo() supplies load average, RAM and uptime. The response converts load and memory units before sending.

```

heishan@localhost:~/IE3090_RemoteOps$ gcc -std=c11 -Wall -Wextra -Wpedantic -O2 -pthread agent_639.c -o agent_639
heishan@localhost:~/IE3090_RemoteOps$ ./controller_639
Connected to Agent at 127.0.0.1:9410
Enter a command, then press Enter.
remoteops> SYSINFO
ERR 003 AUTH_REQUIRED SID:9360
remoteops> AUTH OPS-0639
OK AUTHENTICATED SID:9360
remoteops> SYSINFO
OK SYSINFO 0.08 3443.56 12671 SID:9360
remoteops> QUIT
OK BYE SID:9360
heishan@localhost:~/IE3090_RemoteOps$ |

```

Figure 15. SYSINFO Execution

SYSINFO requires authentication and then returns load, memory and uptime with SID:9360.

3.5 Process Snapshot

```

heishan@localhost:~/IE3090_R  X  heishan@localhost:~/IE3090_  X  Window
static int send_process_list(int fd)
{
    DIR *directory = opendir("/proc");
    if (directory == NULL) {
        return reply(fd, "ERR 008 LISTPROC_FAILED");
    }

    char message[MAX_LINE - 64] = "OK PROCS ";
    size_t used = strlen(message);
    unsigned count = 0;
    struct dirent *entry;

    while (count < 20 && (entry = readdir(directory)) != NULL) {
        if (!is_pid_directory(entry->d_name)) {
            continue;
        }

        char path[128];
        snprintf(path, sizeof(path),
                 "/proc/%.20s/comm", entry->d_name);

        FILE *file = fopen(path, "r");
        if (file == NULL) {
            continue;
        }

        char name[64];
        if (fgets(name, sizeof(name), file) == NULL) {
            fclose(file);
            continue;
        }
        fclose(file);

        name[strcspn(name, "\r\n")] = '\0';
heishan@localhost:~/IE3090_RemoteOps$ |

```

Figure 16. LISTPROC Traversal

The handler opens /proc and filters numeric PID directories before reading process names.

```
heishan@localhost:~/IE3090_R × heishan@localhost:~/IE3090_ × Windo

for (size_t i = 0; name[i] != '\0'; ++i) {
    unsigned char character = (unsigned char)name[i];
    if (!isalnum(character) &&
        character != '_' &&
        character != '-' &&
        character != '.') {
        name[i] = '_';
    }
}

char item[128];
int length = snprintf(item, sizeof(item),
    "%s%s/%.20s",
    count == 0 ? "" : ",",
    name, entry->d_name);

if (length < 0 ||
    (size_t)length >= sizeof(item) ||
    used + (size_t)length >= sizeof(message)) {
    break;
}

memcpy(message + used, item, (size_t)length + 1);
used += (size_t)length;
++count;
}

closedir(directory);

if (count == 0) {
    return reply(f, "ERR 008 LISTPROC_FAILED");
}
return reply(f, message);
}
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 17. LISTPROC Output Construction

Unreadable or exited processes are skipped. Names are sanitised, and the bounded response stops at the chosen process limit.

```
heishan@localhost:~/IE3090_RemoteOps$ nano agent_639.c
heishan@localhost:~/IE3090_RemoteOps$ gcc -std=c11 -Wall -Wextra -Wpedantic -O2 -pthread agent_639.c -o agent_639
heishan@localhost:~/IE3090_RemoteOps$ ./controller_639
Connected to Agent at 127.0.0.1:9410
Enter a command, then press Enter.
remoteops> LISTPROC
ERR 003 AUTH_REQUIRED SID:9360
remoteops> AUTH OPS-0639
OK AUTHENTICATED SID:9360
remoteops> LISTPROC
OK PROCS systemd/1,kthreadd/2,pool_workqueue_release/3,kworker_R-rcu_gp/4,kworker_R-sync_wq/5,kworker_R-kvfree_rcu_reclaim/6,kworker_R-slub_flushwq/7,kworker_R-netns/8,kworker_0-0H-events_highpri/11,kworker_R-mm_percpu_wq/14,ksoftirqd_0/15,rcu_preempt/16,rcu_exp_par_gp_kthread_worker_0/17,rcu_exp_gp_kthread_worker/18,migration_0/19,idle_inject_0/20,cpuhp_0/21,cpuhp_1/22,idle_inject_1/23,migration_1/24 SID:9360
remoteops> SYSINFO
OK SYSINFO 0.14 3447.39 14169 SID:9360
remoteops> QUIT
OK BYE SID:9360
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 18. LISTPROC Execution

The authenticated process snapshot contains readable process names and PIDs; unauthenticated access is rejected.

3.6 Restricted Command Execution

```
/* Exact allow-list: no shell or arbitrary command execution. */
static int send_exec_result(int fd, const char *command)
{
    char output[512];
    int length = -1;

    if (strcmp(command, "DATE") == 0) {
        time_t now = time(NULL);
        struct tm local_time;

        if (now != (time_t)-1 &&
            localtime_r(&now, &local_time) != NULL) {
            size_t result = strftime(output, sizeof(output),
                                     "%Y-%m-%d %H:%M:%S %z",
                                     &local_time);

            if (result > 0) {
                length = (int)result;
            }
        }
    } else if (strcmp(command, "UPTIME") == 0) {
        struct sysinfo information;
        if (sysinfo(&information) == 0) {
            length = snprintf(output, sizeof(output),
                              "%ld seconds", information.uptime);
        }
    } else if (strcmp(command, "DISKFREE") == 0) {
        struct statvfs information;
        if (statvfs(".", &information) == 0) {
            double available =
                (double)information.f_bavail *
                (double)information.f_frsize /
                (1024.0 * 1024.0);

            length = snprintf(output, sizeof(output),
                              "%.2f MB available on project filesystem",
                              available);
        }
    }
    heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 19. EXEC: Time, Uptime and Disk

Exact DATE, UPTIME and DISKFREE commands obtain values through C APIs without invoking a shell.

```
} else if (strcmp(command, "HOSTNAME") == 0) {
    struct utsname information;
    if (uname(&information) == 0) {
        length = snprintf(output, sizeof(output),
                          "%s", information.nodename);
    }
} else if (strcmp(command, "WHOAMI") == 0) {
    struct passwd user;
    struct passwd *result = NULL;
    char buffer[16384];

    int error = getpwuid_r(geteuid(), &user,
                          buffer, sizeof(buffer), &result);
    if (error == 0 && result != NULL) {
        length = snprintf(output, sizeof(output),
                          "%s", result->pw_name);
    }
} else {
    return reply(fd, "ERR 002 COMMAND_NOT_ALLOWED");
}

if (length < 0 || (size_t)length >= sizeof(output)) {
    return reply(fd, "ERR 009 EXEC_FAILED");
}

for (size_t i = 0; output[i] != '\0'; ++i) {
    unsigned char character = (unsigned char)output[i];
    if (character < 32 || character == 127) {
        output[i] = ' ';
    }
}

char message[640];
int result = snprintf(message, sizeof(message),
                      "OK EXEC_RESULT %s", output);

if (result < 0 || (size_t)result >= sizeof(message)) {
    return reply(fd, "ERR 009 EXEC_FAILED");
}
return reply(fd, message);
}
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 20. EXEC: Hostname, User and Rejection

HOSTNAME and WHOAMI use system/user APIs. All other names return COMMAND_NOT_ALLOWED; results are sanitised.

```
heishan@localhost:~$ cd ~/IE3090_RemoteOps
heishan@localhost:~/IE3090_RemoteOps$ ./controller_639
Connected to Agent at 127.0.0.1:9410
Enter a command, then press Enter.
remoteops> EXEC DATE
ERR 003 AUTH_REQUIRED SID:9360
remoteops> AUTH OPS-0639
OK AUTHENTICATED SID:9360
remoteops> EXEC DATE
OK EXEC_RESULT 2026-10-04 01:01:40 +0530 SID:9360
remoteops> EXEC UPTIME
OK EXEC_RESULT 727 seconds SID:9360
remoteops> EXEC DISKFREE
OK EXEC_RESULT 26059.66 MB available on project filesystem SID:9360
remoteops> EXEC HOSTNAME
OK EXEC_RESULT localhost.localdomain SID:9360
remoteops> EXEC WHOAMI
OK EXEC_RESULT heishan SID:9360
remoteops> |
```

Figure 21. Allowed EXEC Commands

The capture records DATE, UPTIME, DISKFREE, HOSTNAME and WHOAMI exchanges using the fixed whitelist.

```
remoteops> EXEC LS
ERR 002 COMMAND_NOT_ALLOWED SID:9360
remoteops> EXEC DATE extra
ERR 002 COMMAND_NOT_ALLOWED SID:9360
remoteops> EXEC DATE;WHOAMI
ERR 002 COMMAND_NOT_ALLOWED SID:9360
remoteops> SYSINFO
OK SYSINFO 0.04 3210.02 840 SID:9360
remoteops> QUIT
OK BYE SID:9360
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 22. Rejected EXEC Requests

Unknown command names, extra arguments and shell punctuation are rejected; the connection remains usable.

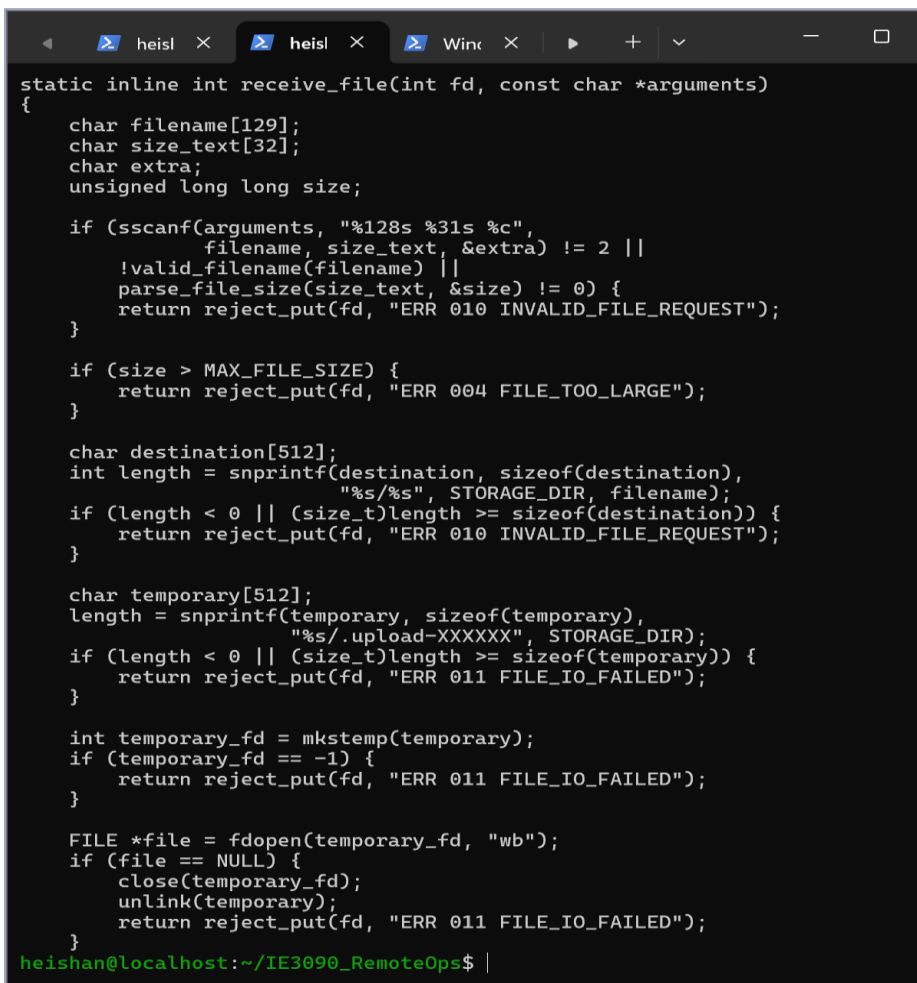
4. File Transfers and Integrity

4.1 Binary Protocol and Safe Storage

PUT accepts simple filenames of at most 128 ASCII characters using letters, digits, underscores, hyphens and dots, with no leading dot. Paths and spaces are rejected. The size parser checks decimal syntax and overflow, then enforces the 100 MiB limit. The Agent receives 4096-byte chunks into a temporary file in its storage directory. A complete upload closes the file and renames it to the destination; an interrupted upload removes its temporary file.

The Controller sends the PUT header immediately followed by the declared payload, without waiting for a ready response. Rejected PUT requests close the session so unread payload cannot be interpreted as commands. GET validates the name and opens a regular file with symlink protection, sends its length header, then sends the exact bytes. The Controller writes a temporary download and renames it after completion. Zero-byte files use the same protocol.

4.2 Upload Implementation



```
static inline int receive_file(int fd, const char *arguments)
{
    char filename[129];
    char size_text[32];
    char extra;
    unsigned long long size;

    if (sscanf(arguments, "%128s %31s %c",
                filename, size_text, &extra) != 2 ||
        !valid_filename(filename) ||
        !parse_file_size(size_text, &size) != 0) {
        return reject_put(fd, "ERR 010 INVALID_FILE_REQUEST");
    }

    if (size > MAX_FILE_SIZE) {
        return reject_put(fd, "ERR 004 FILE_TOO_LARGE");
    }

    char destination[512];
    int length = snprintf(destination, sizeof(destination),
                        "%s/%s", STORAGE_DIR, filename);
    if (length < 0 || (size_t)length >= sizeof(destination)) {
        return reject_put(fd, "ERR 010 INVALID_FILE_REQUEST");
    }

    char temporary[512];
    length = snprintf(temporary, sizeof(temporary),
                    "%s/.upload-XXXXXX", STORAGE_DIR);
    if (length < 0 || (size_t)length >= sizeof(temporary)) {
        return reject_put(fd, "ERR 011 FILE_IO_FAILED");
    }

    int temporary_fd = mkstemp(temporary);
    if (temporary_fd == -1) {
        return reject_put(fd, "ERR 011 FILE_IO_FAILED");
    }

    FILE *file = fdopen(temporary_fd, "wb");
    if (file == NULL) {
        close(temporary_fd);
        unlink(temporary);
        return reject_put(fd, "ERR 011 FILE_IO_FAILED");
    }
}
```

heishan@localhost:~/IE3090_RemoteOps\$

Figure 23. PUT Request Validation

The PUT handler parses filename and declared size, checks validity and the file limit, and rejects malformed or oversized requests.

```
log_event(fd, "PUT_ACCEPTED",
          "filename=%s expected_bytes=%llu", filename, size);

unsigned long long remaining = size;
unsigned long long received = 0;
unsigned char buffer[FILE_CHUNK_SIZE];

while (remaining > 0) {
    size_t chunk = remaining > sizeof(buffer) ?
                sizeof(buffer) : (size_t)remaining;

    if (recv_exact(fd, buffer, chunk) == -1) {
        log_event(fd, "PUT_FAILED",
                  "filename=%s completed_chunk_bytes=%llu expected_byte
s=%llu reason=connection",
                  filename, received, size);
        fclose(file);
        unlink(temporary);
        return -1;
    }

    if (fwrite(buffer, 1, chunk, file) != chunk) {
        log_event(fd, "PUT_FAILED",
                  "filename=%s reason=write", filename);
        fclose(file);
        unlink(temporary);
        return reject_put(fd, "ERR 011 FILE_IO_FAILED");
    }

    remaining -= chunk;
    received += chunk;
}
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 24. PUT Payload Reception

A temporary upload receives the exact declared length in bounded chunks. Receive/write failures close and unlink incomplete storage.

```
if (fclose(file) != 0) {
    unlink(temporary);
    return reject_put(fd, "ERR 011 FILE_IO_FAILED");
}

if (rename(temporary, destination) == -1) {
    unlink(temporary);
    return reject_put(fd, "ERR 011 FILE_IO_FAILED");
}

log_event(fd, "PUT_COMPLETE",
          "filename=%s bytes=%llu storage=%s",
          filename, size, destination);

char message[256];
snprintf(message, sizeof(message),
          "OK FILE_RECEIVED %s", filename);
return file_reply(fd, message);
}
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 25. PUT Completion

Successful close and rename publish the destination file; the response and completion event are recorded.

4.3 Download Implementation

```
static inline int send_file(int fd, const char *filename)
{
    if (!valid_filename(filename)) {
        return file_reply(fd, "ERR 010 INVALID_FILE_REQUEST");
    }

    char path[512];
    int length = snprintf(path, sizeof(path),
                          "%s/%s", STORAGE_DIR, filename);
    if (length < 0 || (size_t)length >= sizeof(path)) {
        return file_reply(fd, "ERR 010 INVALID_FILE_REQUEST");
    }

    int file_fd = open(path, O_RDONLY | O_NOFOLLOW | O_NONBLOCK);
    if (file_fd == -1) {
        log_event(fd, "GET_REJECTED",
                  "filename=%s reason=unavailable", filename);
        return file_reply(fd, "ERR 005 FILE_NOT_FOUND");
    }

    struct stat information;
    if (fstat(file_fd, &information) == -1 ||
        !S_ISREG(information.st_mode) ||
        information.st_size < 0) {
        close(file_fd);
        return file_reply(fd, "ERR 005 FILE_NOT_FOUND");
    }

    unsigned long long size =
        (unsigned long long)information.st_size;

    if (size > MAX_FILE_SIZE) {
        close(file_fd);
        return file_reply(fd, "ERR 004 FILE_TOO_LARGE");
    }
}
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 26. GET File Validation

The GET handler validates a simple filename, opens the requested storage file and verifies regular-file metadata.

```
FILE *file = fdopen(file_fd, "rb");
if (file == NULL) {
    close(file_fd);
    return file_reply(fd, "ERR 011 FILE_IO_FAILED");
}

char message[256];
snprintf(message, sizeof(message),
         "OK FILE_SEND %s %llu", filename, size);

if (file_reply(fd, message) == -1) {
    fclose(file);
    return -1;
}

log_event(fd, "GET_ACCEPTED",
         "filename=%s expected_bytes=%llu", filename, size);

unsigned long long remaining = size;
unsigned long long sent = 0;
unsigned char buffer[FILE_CHUNK_SIZE];

while (remaining > 0) {
    size_t chunk = remaining > sizeof(buffer) ?
        sizeof(buffer) : (size_t)remaining;

    if (fread(buffer, 1, chunk, file) != chunk) {
        log_event(fd, "GET_FAILED",
            "filename=%s completed_chunk_bytes=%llu reason=read",
            filename, sent);
        fclose(file);
        return -1;
    }

    if (send_all(fd, buffer, chunk) == -1) {
        log_event(fd, "GET_FAILED",
            "filename=%s completed_chunk_bytes=%llu reason=connec",
            filename, sent);
        fclose(file);
        return -1;
    }

    remaining -= chunk;
    sent += chunk;
}

fclose(file);
log_event(fd, "GET_COMPLETE",
         "filename=%s bytes=%llu", filename, sent);
return 0;
}

heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 27. GET Header and Payload

The response header identifies filename and exact size before buffered reads and send_all() deliver the payload.

```
if (fread(buffer, 1, chunk, file) != chunk) {
    log_event(fd, "GET_FAILED",
        "filename=%s completed_chunk_bytes=%llu reason=read",
        filename, sent);
    fclose(file);
    return -1;
}

if (send_all(fd, buffer, chunk) == -1) {
    log_event(fd, "GET_FAILED",
        "filename=%s completed_chunk_bytes=%llu reason=connec",
        filename, sent);
    fclose(file);
    return -1;
}

remaining -= chunk;
sent += chunk;
}

fclose(file);
log_event(fd, "GET_COMPLETE",
         "filename=%s bytes=%llu", filename, sent);
return 0;
}

heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 28. GET Completion

The remaining transfer path checks errors, closes the file and records the completed byte count.

4.4 Transfer Execution and Content Integrity

The original, stored and downloaded copies were compared using SHA-256. Matching hashes provide evidence that the tested file contents were preserved. Text, 65536-byte binary and zero-byte files were covered.

```
heishan@localhost:~/IE3090_RemoteOps$ gcc -std=c11 -Wall -Wextra -Wpedantic controller_639.c -o controller_639
heishan@localhost:~/IE3090_RemoteOps$ printf 'RemoteOps file transfer test for IT24100639\n' > sample.txt
heishan@localhost:~/IE3090_RemoteOps$ ./controller_639
Connected to Agent at 127.0.0.1:9410
Authenticate first: AUTH OPS-0639
Upload: PUT filename
Download: GET filename
remoteops> GET sample.txt
ERR 003 AUTH_REQUIRED SID:9360
remoteops> AUTH OPS-0639
OK AUTHENTICATED SID:9360
remoteops> PUT sample.txt
OK FILE_RECEIVED sample.txt SID:9360
remoteops> GET sample.txt
OK FILE_SEND sample.txt 44 SID:9360
Saved 44 bytes to downloads/sample.txt
remoteops> GET missing_639.txt
ERR 005 FILE_NOT_FOUND SID:9360
remoteops> |
```

Figure 29. Text Upload and Download

The Controller exchanges PUT and GET for sample.txt and reports transfer success. Agent replies carry SID:9360.

```
heishan@localhost:~/IE3090_RemoteOps$ gcc -std=c11 -Wall -Wextra -Wpedantic controller_639.c -o controller_639
heishan@localhost:~/IE3090_RemoteOps$ printf 'RemoteOps file transfer test for IT24100639\n' > sample.txt
heishan@localhost:~/IE3090_RemoteOps$ ./controller_639
Connected to Agent at 127.0.0.1:9410
Authenticate first: AUTH OPS-0639
Upload: PUT filename
Download: GET filename
remoteops> GET sample.txt
ERR 003 AUTH_REQUIRED SID:9360
remoteops> AUTH OPS-0639
OK AUTHENTICATED SID:9360
remoteops> PUT sample.txt
OK FILE_RECEIVED sample.txt SID:9360
remoteops> GET sample.txt
OK FILE_SEND sample.txt 44 SID:9360
Saved 44 bytes to downloads/sample.txt
remoteops> GET missing_639.txt
ERR 005 FILE_NOT_FOUND SID:9360
remoteops> QUIT
OK BYE SID:9360
heishan@localhost:~/IE3090_RemoteOps$ sha256sum sample.txt agentfiles/IT24100639/sample.txt downloads/sample.txt
0ec7227ab2a317f840faaac336e6220fd5a52504310d8256a5af071a36ed7016 sample.txt
0ec7227ab2a317f840faaac336e6220fd5a52504310d8256a5af071a36ed7016 agentfiles/IT24100639/sample.txt
0ec7227ab2a317f840faaac336e6220fd5a52504310d8256a5af071a36ed7016 downloads/sample.txt
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 30. Text Transfer Integrity

SHA-256 hashes of the original, Agent copy and downloaded copy match.

```
heishan@localhost:~/IE3090_R × heishan@localhost:~/IE3090_ × + v
c543149671905304be9318804ed8ea16ed4650d089a1ae4b607891eaa5c61a99 binary_test.bin
c543149671905304be9318804ed8ea16ed4650d089a1ae4b607891eaa5c61a99 agentfiles/IT24100639/binary_test.bin
c543149671905304be9318804ed8ea16ed4650d089a1ae4b607891eaa5c61a99 downloads/binary_test.bin
PASS: downloaded binary matches original
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 31. Binary Transfer Integrity

The 65536-byte binary file transfers successfully and its three SHA-256 hashes match.

```
heishan@localhost:~/IE3090_R X heishan@localhost:~/IE3090_ X + v
c543149671905304be9318804ed8ea16ed4650d089a1ae4b607891eaa5c61a99 binary_test.bin
c543149671905304be9318804ed8ea16ed4650d089a1ae4b607891eaa5c61a99 agentfiles/IT24100639/binary_test.bin
c543149671905304be9318804ed8ea16ed4650d089a1ae4b607891eaa5c61a99 downloads/binary_test.bin
PASS: downloaded binary matches original
heishan@localhost:~/IE3090_RemoteOps$ touch empty_test.txt
heishan@localhost:~/IE3090_RemoteOps$ ./controller_639
Connected to Agent at 127.0.0.1:9410
Authenticate: AUTH OPS-0639
Upload: PUT filename
Download: GET filename
Monitoring: MONITOR START 9639
Stop monitoring: MONITOR STOP
remoteops> AUTH OPS-0639
OK AUTHENTICATED SID:9360
remoteops> PUT empty_test.txt
OK FILE_RECEIVED empty_test.txt SID:9360
remoteops> GET empty_test.txt
OK FILE_SEND empty_test.txt 0 SID:9360
Saved 0 bytes to downloads/empty_test.txt
remoteops> QUIT
OK BYE SID:9360
heishan@localhost:~/IE3090_RemoteOps$ wc -c empty_test.txt agentfiles/IT24100639/empty_test.txt downloads/empty_test.txt
0 empty_test.txt
0 agentfiles/IT24100639/empty_test.txt
0 downloads/empty_test.txt
0 total
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 32. Empty File Transfer

PUT and GET accept zero bytes and produce a zero-byte downloaded file.

```
heishan@localhost:~/IE3090_R X heishan@localhost:~/IE3090_ X + v
heishan@localhost:~/IE3090_RemoteOps$ pwd
/home/heishan/IE3090_RemoteOps
heishan@localhost:~/IE3090_RemoteOps$ ls -ld agentfiles/IT24100639
ls -lh agentfiles/IT24100639
ls -lh remoteops_IT24100639.log
drwxr-xr-x. 2 heishan heishan 89 Oct  4 23:40 agentfiles/IT24100639
total 72K
-rw-----. 1 heishan heishan 64K Oct  4 16:08 binary_test.bin
-rw-----. 1 heishan heishan  0 Oct  4 16:14 empty_test.txt
-rw-----. 1 heishan heishan  28 Oct  4 16:00 log_test.txt
-rw-----. 1 heishan heishan  44 Oct  4 01:47 sample.txt
-rw-r--r--. 1 heishan heishan 26K Oct  5 12:05 remoteops_IT24100639.log
heishan@localhost:~/IE3090_RemoteOps$ |
```

Figure 33. Personalised Storage and Log

The directory listing shows agentfiles/IT24100639 with binary_test.bin, empty_test.txt, log_test.txt and sample.txt, plus the correctly named runtime log.

4.5 Rejected and Interrupted Transfers

Direct socket tests exercise requests the interactive Controller would normally reject. They check the Agent independently, including oversized declarations and disconnects during upload.

```

heishan@localhost:~/IE3090_RemoteOps$ ./controller_639
Connected to Agent at 127.0.0.1:9410
Authenticate: AUTH OPS-0639
Upload: PUT filename
Download: GET filename
Monitoring: MONITOR START 9639
Stop monitoring: MONITOR STOP
remoteops> AUTH OPS-0639
OK AUTHENTICATED SID:9360
remoteops> GET ../remoteops_config.h
ERR 010 INVALID_FILE_REQUEST SID:9360
remoteops> GET /etc/passwd
ERR 010 INVALID_FILE_REQUEST SID:9360
remoteops> GET missing_639_test.txt
ERR 005 FILE_NOT_FOUND SID:9360
remoteops> SYSINFO
OK SYSINFO 0.12 3588.66 11277 SID:9360
remoteops> |

```

Figure 34. Invalid GET and Missing File

Traversal and absolute paths produce ERR 010; a missing file produces ERR 005. SYSINFO afterward confirms TCP continuity.

```

heishan@localhost:~/IE3090_R  X  heishan@localhost:~/IE3090_  X  +  v
heishan@localhost:~/IE3090_RemoteOps$ python3 test_upload_errors.py
Agent response: ERR 004 FILE_TOO_LARGE SID:9360
PASS: Upload above 100 MiB rejected; connection closed; no file created.
PASS: Interrupted upload removed; no incomplete final or temporary file.
Agent response: OK SYSINFO 0.00 3617.12 11877 SID:9360
PASS: Agent accepts a new connection and responds after both tests.

ALL THREE TESTS PASSED
heishan@localhost:~/IE3090_RemoteOps$ |

```

Figure 35. Upload Error Tests

The direct socket script checks oversize rejection, interrupted-upload cleanup and Agent availability afterward; all three checks pass.

5. UDP Monitoring and Session Cleanup

5.1 Per-Session UDP Lifecycle

A UDP monitoring datagram has the form SYSINFO <load> <memory_MiB> <uptime_seconds> SID:9360 followed by a newline. Its destination IP comes from getpeername() on the established TCP connection, not an arbitrary address supplied in the command. Sessions use independent workers and requested UDP ports. MONITOR STOP, QUIT, failed authentication and TCP disconnect stop the affected worker. UDP delivery is best effort; loss does not imply TCP command failure.

5.2 Monitoring Worker and Stop Coordination



```
/* Send SYSINFO-style UDP datagrams at a regular interval. */
static inline void *monitor_worker(void *argument)
{
    struct monitor_state *state = argument;

    pthread_mutex_lock(&state->mutex);

    while (!state->stop) {
        struct sysinfo information;

        if (sysinfo(&information) == 0) {
            double cpu_load =
                (double)information.loads[0] / 65536.0;

            double memory_used =
                ((double)information.totalram -
                 (double)information.freeram) *
                (double)information.mem_unit /
                (1024.0 * 1024.0);

            char datagram[256];
            int length = snprintf(
                datagram, sizeof(datagram),
                "SYSINFO %.2f %.2f %ld %s\n",
                cpu_load, memory_used,
                information.uptime, SID_TAG);

            if (length > 0 &&
                (size_t)length < sizeof(datagram)) {
                ssize_t sent;

                do {
                    sent = sendto(
                        state->udp_fd,
                        datagram, (size_t)length,
                        MSG_DONTWAIT,
                        (struct sockaddr *)&state->destination,
                        sizeof(state->destination));
                } while (sent < 0 && errno == EINTR);
            }
        }
    }
}
```

heishan@localhost:~/IE3090_RemoteOps\$ |

Figure 36. UDP Monitoring Worker

The session worker gathers SYSINFO values, constructs a SID-tagged datagram and sends it to the session destination.

```

    }

    struct timespec deadline;

    if (clock_gettime(CLOCK_REALTIME, &deadline) == -1) {
        break;
    }

    deadline.tv_sec += MONITOR_INTERVAL_SEC;

    /* STOP wakes this wait immediately. */
    while (!state->stop) {
        int error = pthread_cond_timedwait(
            &state->condition, &state->mutex, &deadline);

        if (error == ETIMEDOUT) {
            break;
        }

        if (error != 0) {
            state->stop = 1;
            break;
        }
    }

    pthread_mutex_unlock(&state->mutex);
    return NULL;
}
heishan@localhost:~/IE3090_RemoteOps$ |

```

Figure 37. UDP Interval and Stop Coordination

The condition-variable wait provides the two-second cadence while allowing a stop request to wake the worker promptly.

```

    } else if (strncmp(line, "MONITOR START ", 14) == 0) {
        unsigned short port;

        if (parse_udp_port(line + 14, &port) != 0) {
            response = "ERR 013 INVALID_UDP_PORT";
        } else if (monitor_start(&monitor, fd, port) != 0) {
            response = "ERR 012 MONITOR_FAILED";
            log_event(fd, "MONITOR", "start failed port=%u",
                (unsigned)port);
        } else {
            response = "OK MONITOR_STARTED";
            log_event(fd, "MONITOR",
                "started port=%u interval_seconds=%d",
                (unsigned)port, MONITOR_INTERVAL_SEC);
        }
    } else if (strcmp(line, "MONITOR STOP") == 0) {
        monitor_stop(&monitor);
        response = "OK MONITOR_STOPPED";
        log_event(fd, "MONITOR", "stopped by command");
    } else if (strcmp(line, "QUIT") == 0) {
        monitor_stop(&monitor);
        reply(fd, "OK BYE");
        break;
    } else {
        response = "ERR 002 COMMAND_NOT_ALLOWED";
    }

    if (reply(fd, response) == -1) {
        break;
    }
}

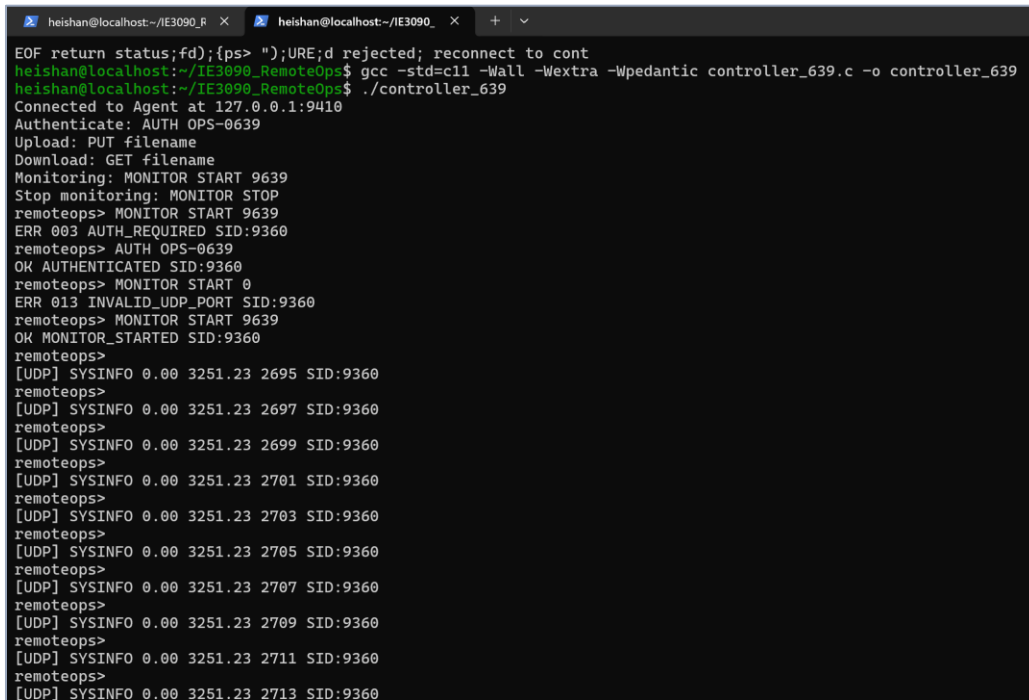
monitor_destroy(&monitor);
log_event(fd, "DISCONNECT", "session ended; monitoring stopped");
close(fd);
return NULL;
}
heishan@localhost:~/IE3090_RemoteOps$ |

```

Figure 38. MONITOR Commands and Disconnect

START validates the port and starts the session worker. STOP and QUIT stop it. Session cleanup destroys monitoring state, logs disconnect and closes TCP.

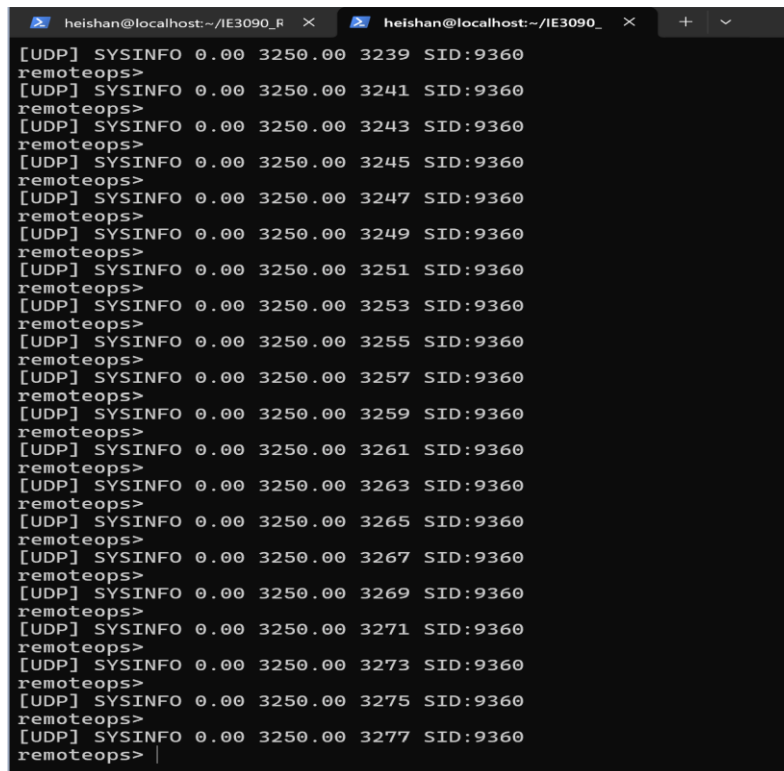
5.3 Monitoring Execution



```
heishan@localhost:~/IE3090_R x heishan@localhost:~/IE3090_ x + v
EOF return status;fd);{ps> ");URE;d rejected; reconnect to cont
heishan@localhost:~/IE3090_RemoteOps$ gcc -std=c11 -Wall -Wextra -Wpedantic controller_639.c -o controller_639
heishan@localhost:~/IE3090_RemoteOps$ ./controller_639
Connected to Agent at 127.0.0.1:9410
Authenticate: AUTH OPS-0639
Upload: PUT filename
Download: GET filename
Monitoring: MONITOR START 9639
Stop monitoring: MONITOR STOP
remoteops> MONITOR START 9639
ERR 003 AUTH_REQUIRED SID:9360
remoteops> AUTH OPS-0639
OK AUTHENTICATED SID:9360
remoteops> MONITOR START 0
ERR 013 INVALID_UDP_PORT SID:9360
remoteops> MONITOR START 9639
OK MONITOR_STARTED SID:9360
remoteops>
[UDP] SYSINFO 0.00 3251.23 2695 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3251.23 2697 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3251.23 2699 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3251.23 2701 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3251.23 2703 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3251.23 2705 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3251.23 2707 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3251.23 2709 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3251.23 2711 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3251.23 2713 SID:9360
```

Figure 39. UDP Monitoring Stream

MONITOR START receives its TCP acknowledgement and the Controller displays periodic SYSINFO datagrams with the SID.



```
heishan@localhost:~/IE3090_R x heishan@localhost:~/IE3090_ x + v
[UDP] SYSINFO 0.00 3250.00 3239 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3241 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3243 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3245 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3247 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3249 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3251 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3253 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3255 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3257 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3259 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3261 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3263 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3265 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3267 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3269 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3271 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3273 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3275 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3277 SID:9360
remoteops> |
```

Figure 40. Further Monitoring Updates

Additional monitoring output shows repeated statistics updates during the active session.

```

remoteops>
[UDP] SYSINFO 0.00 3250.00 3325 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.00 3327 SID:9360
remoteops>
[UDP] SYSINFO 0.00 3250.24 3329 SID:9360
remoteops> MONITOR STOP
[UDP] SYSINFO 0.00 3250.24 3331 SID:9360
remoteops>
OK MONITOR_STOPPED SID:9360
remoteops> SYSINFO
OK SYSINFO 0.07 3250.49 3439 SID:9360
remoteops> |

```

Figure 41. Monitoring Stop and TCP Continuity

MONITOR STOP ends updates, followed by a successful SYSINFO command on the TCP connection.

5.4 Independent Monitoring Sessions

The direct socket test used five authenticated TCP clients and two UDP listeners. STOP, QUIT and disconnect affected the relevant session while the other sessions remained usable. Queued datagrams were discarded before checking for silence over three seconds.

```

heishan@localhost:~/IE3090_R × heishan@localhost:~/IE3090_ × + v
heishan@localhost:~/IE3090_RemoteOps$ python3 test_monitor_sessions.py
PASS: Five authenticated clients remain connected and answer SYSINFO.
PASS: Two client sessions receive valid UDP statistics on separate ports.
PASS: MONITOR STOP stops only its own session; TCP still works.
PASS: QUIT stops that session's UDP; the other session continues.
PASS: Closing TCP without QUIT also stops that session's UDP.
PASS: The remaining three clients continue working.

ALL MONITOR SESSION TESTS PASSED
heishan@localhost:~/IE3090_RemoteOps$ |

```

Figure 42. Monitoring Isolation Tests

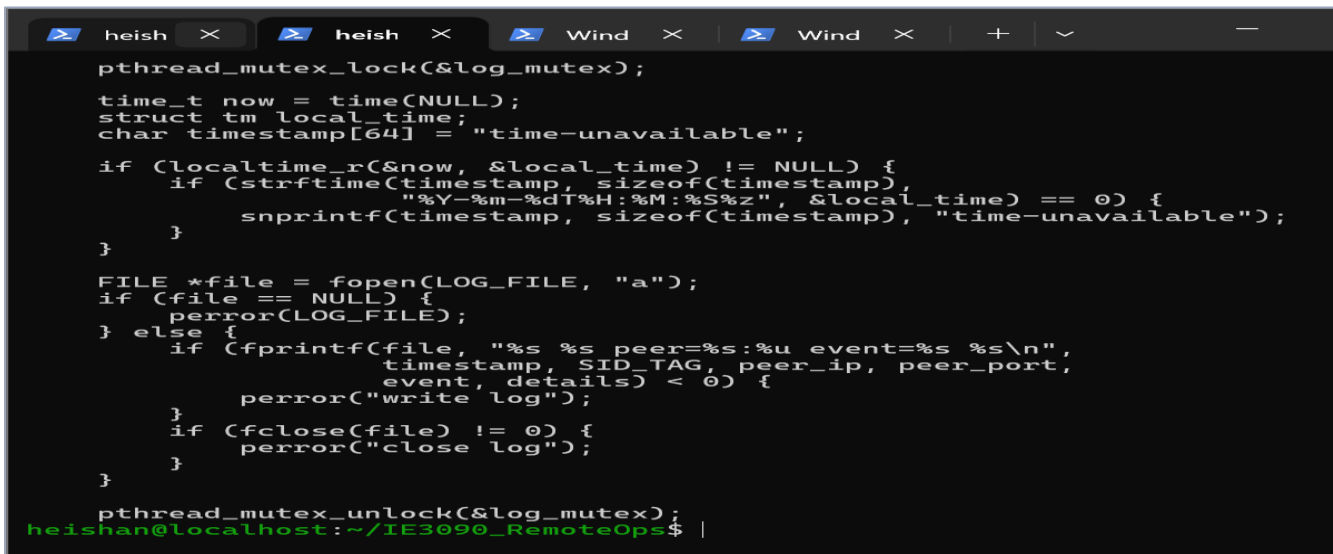
Five clients remain functional while two UDP sessions are independently stopped, quit or disconnected. The silence check exceeds the two-second interval.

6. Logging and Error Handling

6.1 Timestamped Event Records

Logs append local timestamps with timezone, SID, peer IP and port, event name and details to `remoteops_IT24100639.log`. Authentication token text is redacted. Control characters in details are sanitised. Events include server start, connect, commands, authentication results, responses, file completion and disconnect. The uploaded log excerpt and figure 44 provide actual evidence; runtime logs are excluded from Git.

A captured log session includes AUTH failure at 15:58:10 +0530, successful authentication at 15:59:57, SYSINFO at 16:00:04, and a 28-byte PUT of `log_test.txt` at 16:00:08 on 4 October. These values describe the captured run; system statistics and times vary between runs.



```
pthread_mutex_lock(&log_mutex);
time_t now = time(NULL);
struct tm local_time;
char timestamp[64] = "time-unavailable";

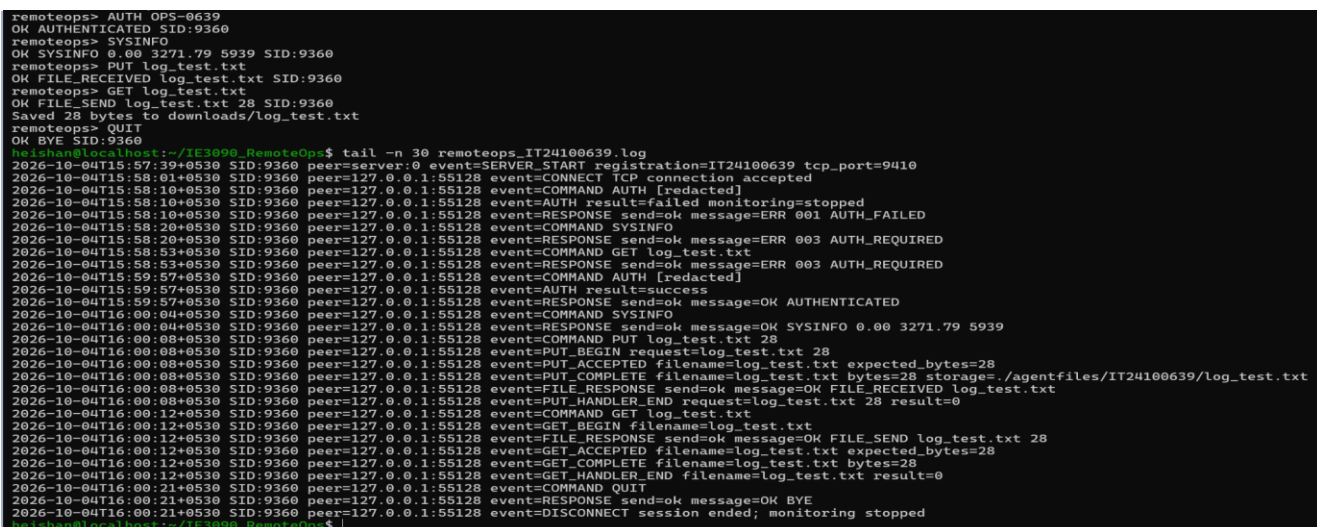
if (localtime_r(&now, &local_time) != NULL) {
    if (strftime(timestamp, sizeof(timestamp),
        "%Y-%m-%dT%H:%M:%S%z", &local_time) == 0) {
        snprintf(timestamp, sizeof(timestamp), "time-unavailable");
    }
}

FILE *file = fopen(LOG_FILE, "a");
if (file == NULL) {
    perror(LOG_FILE);
} else {
    if (fprintf(file, "%s %s peer=%s:%u event=%s %s\n",
        timestamp, SID_TAG, peer_ip, peer_port,
        event, details) < 0) {
        perror("write log");
    }
    if (fclose(file) != 0) {
        perror("close log");
    }
}

pthread_mutex_unlock(&log_mutex);
heishan@localhost:~/IE3090_RemoteOps$
```

Figure 43. Logging Implementation

A mutex protects timestamp formatting and append writes. Entries contain the SID, peer address, event and details.



```
remoteops> AUTH OPS-0639
OK AUTHENTICATED SID:9360
remoteops> SYSINFO
OK SYSINFO 0.00 3271.79 5939 SID:9360
remoteops> PUT log_test.txt
OK FILE_RECEIVED log_test.txt SID:9360
remoteops> GET log_test.txt
OK FILE_SEND log_test.txt 28 SID:9360
Saved 28 bytes to downloads/log_test.txt
remoteops> QUIT
OK BYE SID:9360
heishan@localhost:~/IE3090_RemoteOps$ tail -n 30 remoteops_IT24100639.log
2026-10-04T15:57:39+0530 SID:9360 peer=server:0 event=SERVER_START registration=IT24100639 tcp_port=9410
2026-10-04T15:58:01+0530 SID:9360 peer=127.0.0.1:55128 event=CONNECT TCP connection accepted
2026-10-04T15:58:10+0530 SID:9360 peer=127.0.0.1:55128 event=COMMAND AUTH [redacted]
2026-10-04T15:58:10+0530 SID:9360 peer=127.0.0.1:55128 event=AUTH result=failed monitoring=stopped
2026-10-04T15:58:10+0530 SID:9360 peer=127.0.0.1:55128 event=RESPONSE send=ok message=ERR 001 AUTH_FAILED
2026-10-04T15:58:20+0530 SID:9360 peer=127.0.0.1:55128 event=COMMAND SYSINFO
2026-10-04T15:58:20+0530 SID:9360 peer=127.0.0.1:55128 event=RESPONSE send=ok message=ERR 003 AUTH_REQUIRED
2026-10-04T15:58:33+0530 SID:9360 peer=127.0.0.1:55128 event=RESPONSE send=ok message=ERR 003 AUTH_REQUIRED
2026-10-04T15:59:57+0530 SID:9360 peer=127.0.0.1:55128 event=COMMAND AUTH [redacted]
2026-10-04T15:59:57+0530 SID:9360 peer=127.0.0.1:55128 event=AUTH result=success
2026-10-04T15:59:57+0530 SID:9360 peer=127.0.0.1:55128 event=RESPONSE send=ok message=OK AUTHENTICATED
2026-10-04T16:00:04+0530 SID:9360 peer=127.0.0.1:55128 event=COMMAND SYSINFO
2026-10-04T16:00:08+0530 SID:9360 peer=127.0.0.1:55128 event=RESPONSE send=ok message=OK SYSINFO 0.00 3271.79 5939
2026-10-04T16:00:08+0530 SID:9360 peer=127.0.0.1:55128 event=COMMAND PUT log_test.txt 28
2026-10-04T16:00:08+0530 SID:9360 peer=127.0.0.1:55128 event=PUT_BEGIN request=log_test.txt 28
2026-10-04T16:00:12+0530 SID:9360 peer=127.0.0.1:55128 event=GET_BEGIN filename=log_test.txt expected_bytes=28
2026-10-04T16:00:12+0530 SID:9360 peer=127.0.0.1:55128 event=PUT_COMPLETE filename=log_test.txt bytes=28 storage=/agentfiles/IT24100639/log_test.txt
2026-10-04T16:00:12+0530 SID:9360 peer=127.0.0.1:55128 event=FILE_RESPONSE send=ok message=OK FILE_RECEIVED log_test.txt
2026-10-04T16:00:12+0530 SID:9360 peer=127.0.0.1:55128 event=PUT_HANDLER_END request=log_test.txt 28 result=0
2026-10-04T16:00:12+0530 SID:9360 peer=127.0.0.1:55128 event=GET_COMPLETE filename=log_test.txt bytes=28
2026-10-04T16:00:12+0530 SID:9360 peer=127.0.0.1:55128 event=GET_HANDLER_END filename=log_test.txt result=0
2026-10-04T16:00:21+0530 SID:9360 peer=127.0.0.1:55128 event=COMMAND QUIT
2026-10-04T16:00:21+0530 SID:9360 peer=127.0.0.1:55128 event=RESPONSE send=ok message=OK BYE
2026-10-04T16:00:21+0530 SID:9360 peer=127.0.0.1:55128 event=DISCONNECT session ended; monitoring stopped
heishan@localhost:~/IE3090_RemoteOps$
```

Figure 44. Timestamped Log Evidence

The correctly named log records timestamped authentication, response, transfer and disconnect events; token contents are redacted.

6.2 Error Behaviour

Recorded errors include AUTH_FAILED (001), COMMAND_NOT_ALLOWED (002), AUTH_REQUIRED (003), FILE_TOO_LARGE (004), FILE_NOT_FOUND (005), LINE_TOO_LONG (006) and INVALID_FILE_REQUEST (010). Invalid GET and missing-file responses leave the connection usable; rejected PUT closes it. Invalid monitoring ports and worker creation failures have explicit errors.

Invalid commands return protocol errors. Invalid GET requests preserve the TCP connection, while rejected PUT requests close it to avoid processing unread file bytes as command text. Failure paths release sockets, close files and remove incomplete uploads where applicable.

7. Build and Testing Summary

Makefile_639 builds agent_639 and controller_639 using gcc -std=c11 -Wall -Wextra -Wpedantic -O2 and -pthread. Figure 45 records a clean rebuild with both GCC commands and no warnings or errors. Figure 3 records Agent startup on port 9410; figure 2 shows the listener and figure 33 proves the storage path and uploaded files.

Manual tests used the C Controller. Python scripts are supplementary direct socket tests for cases the Controller normally prevents or cannot deliberately generate. They are not the Agent or Controller implementation. Splitting application sends checks stream handling; it does not claim control over actual TCP packet segmentation.

Test and evidence	Expected	Observed
AUTH and access control Figures 13, 15, 18	Reject before AUTH; accept correct token	Failed token and unauthenticated commands rejected; successful AUTH and QUIT
Five C Controllers Figures 6, 7	Five simultaneous sessions	Five established TCP connections; five AUTH and BYE responses
SYSINFO and LISTPROC Figures 15, 18	Statistics and process snapshot with SID	Authenticated outputs returned with SID:9360
EXEC whitelist Figures 21, 22	Five allowed commands; reject others	All allowed commands returned; LS, extra arguments and semicolon string rejected
Text PUT GET Figures 29, 30	Store and download unchanged file	44-byte transfer completed; three SHA-256 hashes matched
Binary and empty files Figures 31, 32	Preserve binary bytes; accept zero length	65536-byte hashes matched; zero-byte transfers completed
GET errors Figure 34	Reject paths/missing file; TCP survives	ERR 010 and ERR 005; subsequent SYSINFO succeeded
Upload error script Figure 35	Reject over 100 MiB; remove incomplete upload	Three checks passed; no final or new temporary upload left
TCP stream script Figure 46	Handle split/combined commands and file boundaries	All three checks passed
UDP start and stop Figures 39, 41	Periodic updates; stop and preserve TCP	Statistics arrived; STOP followed by successful TCP SYSINFO
Monitoring isolation script Figure 42	Independent sessions; STOP QUIT EOF cleanup	Five clients worked; two UDP sessions isolated; all checks passed

Test and evidence	Expected	Observed
Logging Figure 44	Timestamp SID peer events and token redaction	AUTH, PUT_COMPLETE, GET_COMPLETE and DISCONNECT recorded

7.1 Final Clean Build

```

heishan@localhost:~/IE3090_R × heishan@localhost:~/IE3090_ × Windows PowerShell ×
heishan@localhost:~/IE3090_RemoteOps$ make -f Makefile_639 clean
rm -f agent_639 controller_639
heishan@localhost:~/IE3090_RemoteOps$ make -f Makefile_639
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 agent_639.c -o agent_639 -pthread
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 controller_639.c -o controller_639 -pthread
heishan@localhost:~/IE3090_RemoteOps$ |

```

Figure 45. Final Clean Build

Both personalised C source files compile through Makefile_639 with C11, warnings, optimisation and pthread support. No diagnostic appears.

7.2 TCP Stream Robustness

The framing tests deliberately split command text across sends, combine several commands in one send and place binary payload bytes between neighbouring commands. These test application stream boundaries; they do not control the packet segmentation chosen by TCP.

```

heishan@localhost:~/IE3090_R × heishan@localhost:~/IE3090_ × + v
heishan@localhost:~/IE3090_RemoteOps$ python3 test_tcp_stream.py
PASS: AUTH and SYSINFO work when split across separate sends.
PASS: Three commands sent together receive separate ordered replies.
PASS: File headers, binary payloads and following commands stay separate.

ALL THREE TCP STREAM TESTS PASSED
heishan@localhost:~/IE3090_RemoteOps$ |

```

Figure 46. TCP Framing Tests

Split commands, combined commands, and binary transfer boundaries pass the three direct socket checks.

8. Incremental Development and AI Assistance

8.1 Meaningful Development Milestones

The uploaded project snapshot is at commit aa2d5c9. The exported history records 16 earlier commits across 2 to 5 October, followed by prompt-log clarification and reflection commits. The table preserves actual commit identifiers and messages; it is not a reconstructed terminal screenshot.

Commit	Date	Development milestone
4332190	2026-10-02	Document personalised settings and initial development setup
d1142d1	2026-10-02	Add personalised configuration and initial TCP Agent listener
49e8962	2026-10-02	Add threaded client sessions, authentication and TCP Controller
4c595bb	2026-10-02	Document successful five-client concurrency test
c30f21e	2026-10-02	Add authenticated SYSINFO using real Linux statistics
f48afa1	2026-10-03	Add authenticated LISTPROC snapshot from proc filesystem
7bba93c	2026-10-04	Add five authenticated allowlisted EXEC commands
b21431c	2026-10-04	Implement authenticated TCP file uploads and downloads
6ccab6a	2026-10-04	Ignore compiled programs and runtime test files
f9256c9	2026-10-04	Add per-client UDP monitoring and controller reception
40a290c	2026-10-04	Add personalised Makefile for Agent and Controller
6539083	2026-10-04	Add timestamped connection command and file transfer logging
c275d64	2026-10-04	test: verify upload limits and interrupted transfer cleanup
487106c	2026-10-04	test: verify TCP command and binary payload framing
5a50866	2026-10-04	test: verify monitoring isolation and session cleanup
b0452f3	2026-10-05	docs: update usage instructions and consolidate design diary
3298962	2026-10-05	Prompt log clarification
aa2d5c9	2026-10-05	docs: add structured reflection on learning and AI use

8.2 Design Diary and Reflection

The design diary records the environment, implementation decisions and testing progress. The earlier detailed notes are retained in docs/development_history.md. The separate reflection describes the AI assistance, practical

work, difficulties and learning. Understanding the C code remains an area where I need more practice, particularly sockets and thread cleanup.

8.3 AI Assistance and Practical Verification

ChatGPT/Codex assisted with code drafts, troubleshooting, test scripts and documentation. I updated the project files in CentOS, compiled and ran the programs, executed tests, checked replies, hashes and logs, captured screenshots and maintained Git commits. The separate prompt log records the assistance and verification. Report organisation was revised with AI support using the existing project and captured evidence.

9. Assumptions, Limitations and Future Improvements

9.1 Operating Assumptions

Both programs are run from the project directory. The personalised Agent storage directory exists and is writable. For remote use, the Agent TCP port and the Controller UDP listening port must be reachable. Commands are case sensitive, filenames use the restricted supported character set and transfers are limited to 100 MiB.

9.2 Implementation Limitations

The authentication token and network traffic are unencrypted. UDP can lose updates. There is no application idle timeout, maximum client thread count or per-file upload lock. Concurrent completed uploads using the same filename may replace each other; the last successful rename determines the stored content. The recorded tests cover observed cases rather than every possible input, resource limit or network failure.

9.3 Optional Extension Status and Future Work

No TLS, snapshot history retrieval, throughput display or compression extension is implemented in this submitted source snapshot. These features must not be inferred from the mandatory tests. Future work could add transfer throughput using an elapsed-time measurement, together with larger-file tests and a clear definition of what the timed interval includes. TLS and connection limits would require additional implementation and testing.

10. Conclusion

The recorded implementation provides authenticated TCP commands, a bounded process snapshot, five restricted EXEC operations, binary uploads and downloads, per-session UDP monitoring and timestamped logging. Five C Controllers connected simultaneously. Additional socket tests checked file-transfer failures, command framing and independent monitoring cleanup.

The included code screenshots explain how the main handlers work, while execution captures document the observed results. The project uses personalised settings for IT24100639 and an incremental Git history. The final submission also needs the matching source archive, Makefile, documentation, prompt log, reflection and sample log. This report documents the verified mandatory implementation without claiming an optional extension or exhaustive testing.